

고급 파이썬 프로그래밍 및 자동화

최종 과제

1. 연구 코드 소개

가. 연구 주제 또는 실험 맥락

현재 엔진 조립 공정에서 볼트를 조립한 후 얻은 시계열 데이터를 분석하여 불량률 판별하는 분류기를 제작 중입니다. 기계에서 데이터를 받아 PC로 저장해주는 소프트웨어를 제조사에서 제공하기 때문에, 이 로우 데이터를 가공하여 정답 레이블과 시계열 데이터를 구분하는 프로그램, 시계열 데이터의 길이를 통일시키는 프로그램, 체결 공정과 해체 공정을 구분하는 프로그램, 훈련 데이터와 시험 데이터로 구분하는 프로그램, 모델을 학습시키는 프로그램 등으로 나누어져 있습니다. 이 중에서 시계열 데이터의 길이를 통일시키는 프로그램을 대상으로 하였습니다.

나. 해당 코드의 입력과 출력

이 프로그램은 레이블 데이터와 시간, 토크, 각도로 구성된 시계열 데이터를 파일에서 읽은 다음 레이블 데이터는 수정 없이 그대로, 시계열 데이터는 길이만 조정된 다음 동일한 구조의 파일로 출력합니다.

다. 전체 연구 파이프라인에서의 위치

이 프로그램은 위에 설명한 것처럼 전처리 단계에 사용됩니다.

라. 해당 코드를 개선 대상으로 선택한 이유

이 코드는 단순 반복 작업을 많이 수행하기 때문에 개선 전후에 실행 속도의 차이가 잘 드러날 것으로 생각되며, 데이터 길이를 수정하는 작업 밖에 하지 않으므로 코드가 공개되어도 보안 문제가 없을 것으로 생각되어 선정하였습니다.

2. 기존 코드 문제점 분석

가. 실행 시간 측면의 문제

cProfile을 사용해 각 함수의 소요 시간을 정리한 결과는 다음과 같습니다.

44,871,446 function calls (44,747,455 primitive calls) in 66.554 seconds

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
21,533	24.155	0.001	24.155	0.001	{method 'astype' of 'numpy.ndarray' objects}
606	18.937	0.031	52.268	0.086	csvs.py:315(_save_chunk)
20,127,329	4.648	0	7.834	0	utf_8_sig.py:31(encode)
20,127,329	3.186	0	3.186	0	{built-in method _codecs.utf_8_encode}
2	2.992	1.496	3.123	1.562	c_parser_wrapper.py:222(read)
2	1.317	0.658	53.588	26.794	csvs.py:305(_save_body)
14	1.188	0.085	1.253	0.09	take.py:120(_take_nd_ndarray)

전체 실행 시간이 1초 이상인 7개의 결과만 표로 정리하였습니다. 1위인 numpy.ndarray.astype 메소드가 전체 실행시간의 36.3%를, 2위인 _save_chunk 메소드가 28.5%를 차지하는 것을 볼 수 있습니다. 3위부터는 실행 시간의 7% 미만을 소요하기 때문에 중요도가 낮은 것으로 보입니다. 코드에서

astype 메소드를 호출한 적이 없고, 2위는 판다스 내부 메소드인 _save_chunk인 것으로 보아 1, 2위 모두 판다스 데이터프레임에서 일어나는 일로 생각됩니다.

나. 메모리 사용 측면의 문제

memory_profiler를 사용하여 각 코드의 메모리 사용량 변화를 정리한 결과는 다음과 같습니다.

truncate_and_pad()

Line #	Mem usage	Increment	Occurrences	Line Contents
26	412.0 MiB	412.0 MiB	1	@profile
29	594.2 MiB	182.1 MiB	1	n_samples = len(grouped)
32	747.7 MiB	153.5 MiB	1	time_vals[:] = np.linspace(0, (target_length - 1) / cfg.sampling_rate_hz, target_length)
35	1573.2 MiB	518.4 MiB	9824	for idx, (i, group) in enumerate(grouped):
47	1573.2 MiB	153.5 MiB	9823	sample_vals[idx] = np.full(target_length, group[cfg.sample].values[0])
51	1573.2 MiB	153.5 MiB	9823	other_arrays[c][idx] = arr
70	1880.1 MiB	307.0 MiB	1	df_dict = {cfg.time_s: time_vals.flatten(), cfg.sample: sample_vals.flatten()}
72	2033.6 MiB	153.5 MiB	1	df_dict[c] = other_arrays[c].flatten()
73	2494.1 MiB	460.5 MiB	1	df_out = pd.DataFrame(df_dict)
74	1961.7 MiB	-532.4 MiB	1	df_out = df_out[df_in.columns]

main()

Line #	Mem usage	Increment	Occurrences	Line Contents
76	147.4 MiB	147.4 MiB	1	@profile
80	412.0 MiB	264.6 MiB	1	df_label, df_series_in = common.load_data_csv(input_folder_path)
84	705.3 MiB	293.3 MiB	1	df_series_out = truncate_and_pad(df_series_in, cfg.target_sample_length, cfg.peak_position)
88	728.9 MiB	23.6 MiB	1	common.save_data_csv(this_file, df_label, df_series_out)

1 MB 이상의 변화가 있었던 행만 나열하였습니다. 메모리를 가장 많이 사용하는 곳은 grouped를 사용하는 for문, 2위는 df_dict를 데이터프레임으로 변환하는 부분, 3위는 flatten하여 딕셔너리에 할당하는 부분, 4~5위는 데이터프레임을 로드하고 저장하는 부분입니다. 시간과 마찬가지로 메모리 사용량도 대부분 판다스 데이터프레임과 관련이 있음을 알 수 있습니다.

다. 자료구조 선택의 문제

판다스는 꽤 무거운 라이브러리이므로 이렇게 대량의 데이터를 처리하기엔 부적합한 것으로 생각됩니다. 좀 더 가벼운 자료구조로의 전환이 필요합니다.

라. 반복 계산 또는 중복 코드

```
grouped = df_in.groupby(cfg.sample)
```

```
for idx, (i, group) in enumerate(grouped):
```

위처럼 판다스 데이터프레임을 그룹으로 나눈 다음 반복문을 돌리는 구조입니다. 판다스는 이런 경우

매번 새로운 데이터프레임 객체를 만들기 때문에 상당히 느려집니다.

마. 함수나 클래스의 책임이 과도하게 섞인 부분

코드가 절차적으로 작성되어 `truncate_and_pad` 함수가 거의 모든 작업을 다 수행합니다.

바. 실험 조건 변경이 어려운 부분

이 코드는 실험 조건 변경과는 큰 관련이 없습니다. 조건을 바꾸어도 여전히 시간, 토크, 각도 데이터가 나올 것이기 때문입니다.

사. 재현성 또는 유지보수성 문제

이 코드는 완전히 결정적으로 동작하므로 재현성 문제는 없습니다. 유지보수성은 절차적 코드의 특성상 상당히 나쁜 편입니다. 코드가 짧기 때문에 두드러지지는 않으나, 객체지향적으로 수정하는 것이 좋을 것입니다.

3. 적용한 최적화 또는 리팩터링 개념

계산 시간 및 메모리 사용 모두 대부분 pandas dataframe에서 발생합니다. 따라서 다음과 같은 개선을 도입하였습니다.

가. 자료구조 및 복잡도 기반 개선

2백 MB인 원본 csv 파일을 한 번에 로드하여 판다스 데이터프레임으로 만들면 약 7백 MB를 차지합니다. 판다스는 자체적으로 많은 메타데이터를 갖고, 해시 맵은 데이터를 담을 수 있는 최소 크기의 1~2 배이기 때문에 생기는 현상으로 생각됩니다. 따라서 루프에서는 `dataFrame.groupby()` 메소드 대신 좀 더 가벼운 넘파이 ndarray 자료구조로 변경합니다.

나. Iterator, Generator, Lazy Evaluation 기반 개선

csv 파일 입출력에는 판다스를 사용하는 것이 매우 편리하므로 판다스 데이터프레임을 유지하는 대신, 한 번에 전체 파일을 로드하지 않고 청크 단위로 로드하여 메모리 사용량을 줄입니다. `pd.read_csv` 메소드는 eager generation 방식으로 동작하여 파일 전체를 데이터프레임으로 바로 반환하지만, `chunksize` 옵션을 사용하는 경우 lazy generation을 사용할 수 있도록 iterator인 `TextFileReader` 오브젝트를 반환합니다.

다. Class 기반 구조 개선 및 단일책임원칙 적용

현재 코드는 절차적으로 작성되어 있어 향후 규모가 커지거나 기능을 추가할 때 유지보수가 어렵습니다. 따라서 객체지향형 코드로 수정합니다.

4. 개선 과정

가. 수정 전 코드 구조

절차적 코드로, 다음과 같은 구조입니다.

- 1) 입력 csv 파일 찾기
- 2) 데이터 로드하여 판다스 데이터프레임 생성
- 3) 샘플 번호 별로 데이터 나누기
- 4) 각 샘플별 토크 피크 위치 탐지 및 정렬
- 5) 각 샘플별 다른 시계열 데이터 정렬
- 6) 출력 csv 파일 저장

나. 수정 후 코드 구조

객체 지향형 코드로, 다음과 같은 구조입니다.

- 1) PreprocessConfig 클래스
데이터클래스로써, 설정값을 저장합니다.

- 2) SeriesProcessor 클래스
시계열 데이터의 처리를 담당합니다.

가) allocate_buffers 메소드
고속 처리를 위해 정확히 필요한 크기의 넘파이 ndarray 변수를 만들어서 반환합니다.

나) build_output_dataframe 메소드
파일 저장을 위해 넘파이 ndarray 자료를 판다스 데이터프레임으로 만듭니다.

다) process_chunk_vectorized 메소드
판다스 데이터프레임 자료를 넘파이 ndarray로 변환한 다음 벡터 연산을 통해 고속으로 처리합니다.

라) calculate_shift_indices 메소드
피크 정렬을 위해 시프트할 값을 계산합니다.

- 3) StreamPipeline 클래스
디스크에서 csv 파일을 읽고 쓰는 작업, 시계열 데이터를 추출하여 SeriesProcessor에 넘겨주는 작업을 담당합니다.

가) calculate_optimal_chunk_size 메소드
고속 처리를 위해 정확히 필요한 크기의 넘파이 ndarray 변수를 만들어서 반환합니다.

나) append_to_csv 메소드
파일 저장을 위해 넘파이 ndarray 자료를 판다스 데이터프레임으로 만듭니다.

다) run 메소드
판다스 데이터프레임 자료를 넘파이 ndarray로 변환한 다음 벡터 연산을 통해 고속으로 처리합니다.

- 4) main 함수

설정값을 읽어오고 StreamPipeline을 실행합니다.

다. 주요 변경 사항

절차적 코드를 객체지향형 코드로 수정하였습니다. 또한 자료구조로 판다스 데이터프레임만을 사용하고 루프도 grouped를 사용하던 것을 넘파이 ndarray를 사용하게 수정하였습니다.

라. 변경한 이유

우선 속도 차이가 큽니다. 판다스 데이터프레임은 슬라이스할 때마다 새로운 객체를 생성하기 때문에 큰 데이터를 다룰 때 메모리 사용량이 많고 느립니다. 이에 비해 넘파이는 배열을 사용하기 때문에 빠르게 계산이 가능합니다.

마. 고려했지만 적용하지 않은 대안

특별히 없습니다.

바. 최적화 과정에서 발생한 trade-off

속도와 메모리 사용량 모두 감소하였습니다. 트레이드 오프는 없는 것으로 생각됩니다.

5. 최적화 결과

시험 환경은 다음과 같습니다.

항목	내용
OS	Windows 10
Python 버전	3.9.13
주요 라이브러리 버전	matplotlib==3.9.4 matplotlib-inline==0.1.7 memory-profiler==0.61.0 numpy==2.0.2 pandas==2.2.3 scipy==1.13.1
CPU 또는 GPU 정보	AMD Ryzen 5 3400G
입력 데이터 크기	201 MB
반복 횟수	3회
random seed 사용 여부	X
warm-up 수행 여부	X

가. 실행 시간 비교

cProfile을 사용하여 실행 시간을 비교하였습니다.

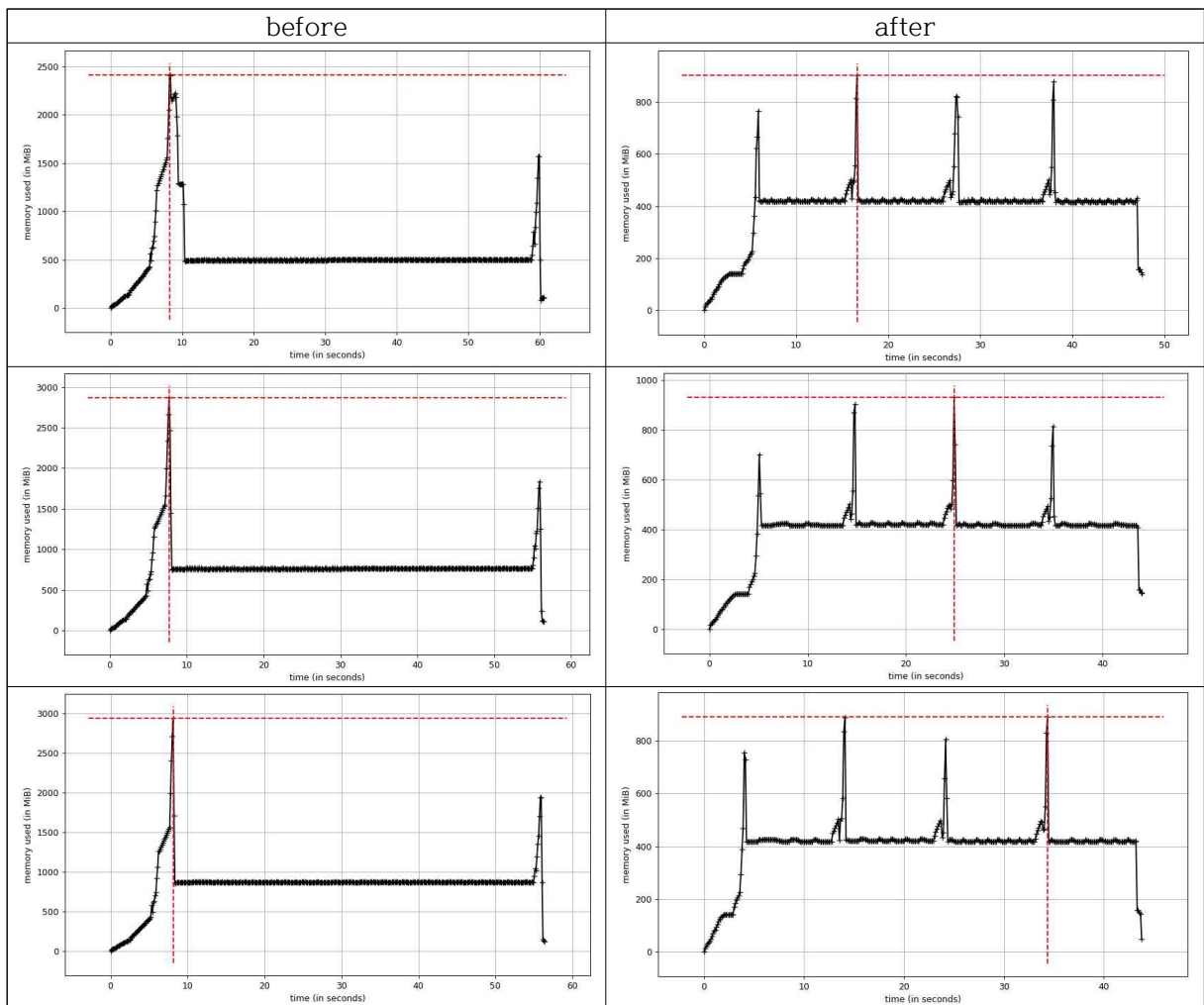
	before	after
1회	67.538 s	45.252 s
2회	63.894 s	44.740 s
3회	62.931 s	44.166 s
평균	64.788 s	44.719 s

평균 시간이 64.8초에서 44.7초로 31% 감소하였습니다.

나. 메모리 사용량 비교

mprof를 사용하여 메모리 사용량을 비교하였습니다.

	before	after
1회	2410.246 MB	902.273 MB
2회	2866.543 MB	930.277 MB
3회	2936.289 MB	889.715 MB
평균	2737.693 MB	907.422 MB



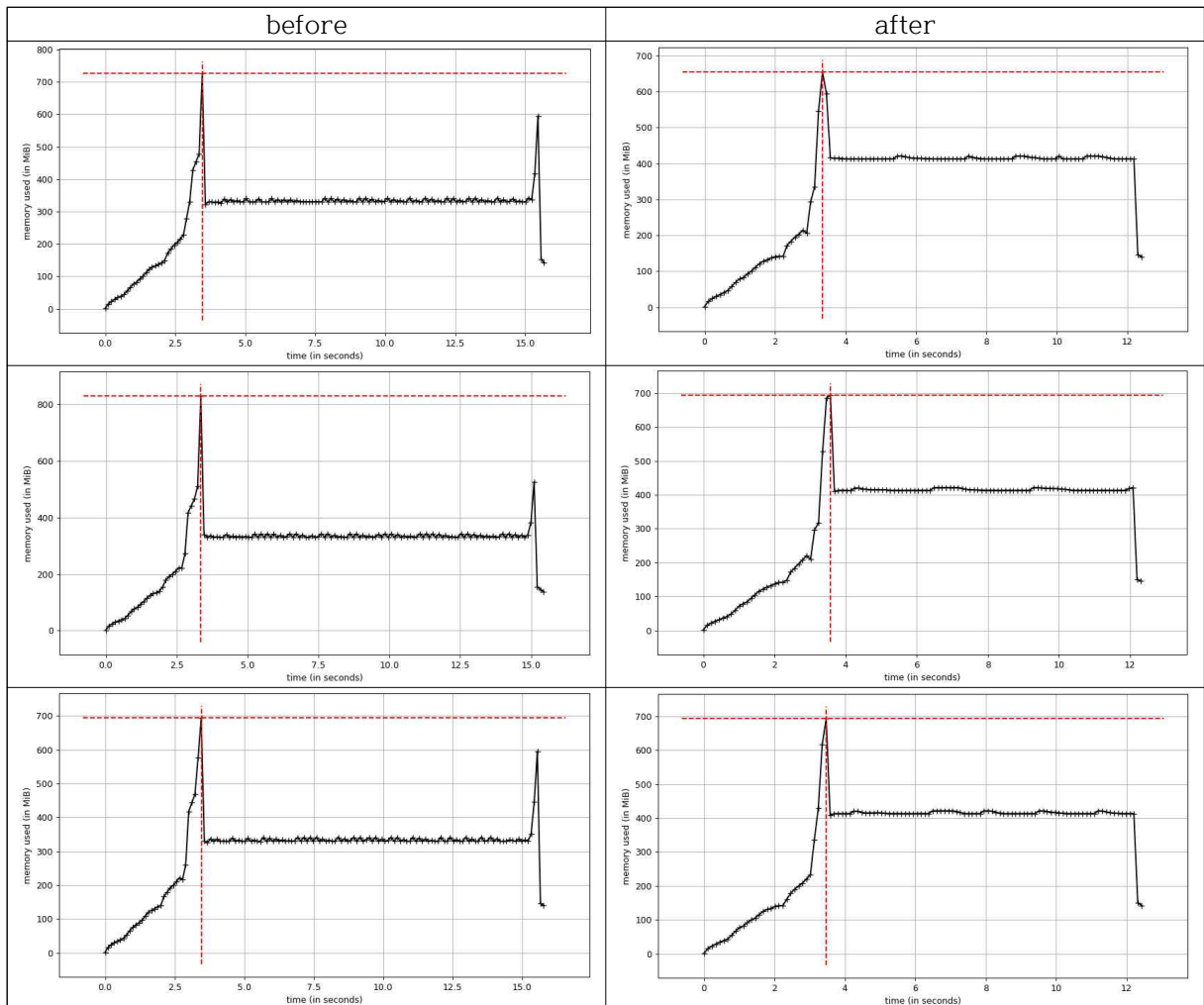
최대 메모리 사용량 평균이 2737 MB에서 907 MB로 67% 감소하였습니다.

다. 입력 크기별 성능 변화

입력 크기가 1/4로 줄었을 때의 실행 시간과 메모리 사용량 비교표입니다.

	before	after
1회	17.131 s	12.637 s
2회	16.996 s	13.213 s
3회	17.479 s	12.873 s
평균	17.202 s	12.908 s

	before	after
1회	725.961 MB	654.590 MB
2회	828.395 MB	692.887 MB
3회	694.062 MB	692.828 MB
평균	749.473 MB	680.102 MB



평균 시간이 17.2초에서 12.9초로 25% 감소, 최대 메모리 사용량 평균이 749 MB에서 680 MB로 9% 감소하였습니다. 작은 데이터셋에선 시간 이득은 큰 차이 없으나 메모리 사용량의 이득은 사라지는 것을 알 수 있었습니다.

라. 코드 구조 변화

코드 구조는 절차적 구조에서 객체지향적으로 바뀌었습니다.

마. 중복 제거 결과

특별히 중복되는 부분은 없습니다.

바. 재사용성 또는 확장성 개선 결과

각 기능이 클래스와 메소드로 분산되어 재사용 및 확장성이 좋아진 것으로 생각합니다.

사. 실험 조건 변경 시 코드 수정 범위 비교

이 코드는 실험 조건이 바뀌어도 수정할 필요가 없습니다.

6. 결과 해석 및 한계

가. 기대한 개선이 실제로 나타났는지

기대한 것처럼 처리 속도가 30% 가량 빨라졌으며, 특히 메모리 사용량은 1/3로 감소하였습니다.

나. 개선 폭이 큰 부분과 그렇지 않은 부분

작은 데이터셋에서는 메모리 사용량이 개선 전후에 큰 차이가 없었습니다. 입력 데이터가 클수록 전체 데이터를 메모리에 적재하는 이전 코드와의 차이가 클 것으로 생각되므로 더 큰 데이터셋에서는 더 크게 유리할 것으로 생각합니다.

실행 시간은 데이터셋이 크건 작건 간에 다소 빨라졌습니다.

다. 성능은 좋아졌지만 코드 복잡도가 증가한 부분

클래스 기반으로 수정하였기 때문에 코드 복잡도는 오히려 감소하였습니다.

라. 메모리는 줄었지만 실행 시간이 늘어난 부분

트레이드 오프는 없었습니다.

마. lazy evaluation 적용으로 생긴 장단점

현재 코드는 입력을 lazy하게 처리하기 위해 파일을 여러 번 나눠서 읽습니다. 일반적인 HDD나 SSD에서는 문제가 없지만, 저장 장치의 특성에 따라 이것이 손해가 될 수도 있습니다. 가령 테이프 레코더처럼 순차 읽기만 지원하는 장비를 여러 프로세스가 접근하는 경우에는 훨씬 긴 대기 시간이 발생할 것입니다.

바. decorator 또는 class 구조 도입으로 생긴 오버헤드

클래스 구조 도입으로 발생한 오버헤드는 판다스 데이터프레임을 넘파이 ndarray로 변경한 부분이 모두 상쇄하고도 남는 것으로 보입니다.

사. 추가로 개선 가능한 부분

이 코드는 짧기 때문에 여기서 추가적인 개선이 필요할 것으로 생각되지는 않습니다. 전처리 프로그램은 이 파일 하나만 있는 것이 아니므로, 다른 전처리 프로그램에도 동일한 수정을 가하면 좋을 것으로 생각합니다.